

Hooks de base : useEffect et effets secondaires

Objectifs de cette section

Après `useState`, vous allez découvrir `useEffect`, un hook indispensable pour gérer les **effets secondaires** : appels API, mise à jour du document, gestion de timers, etc.

À la fin de cette section, vous serez capable de :

- Expliquer ce qu'est un effet secondaire dans une application React.
 - Utiliser `useEffect` pour déclencher un comportement au montage d'un composant.
 - Utiliser `useEffect` pour réagir à un changement d'état.
 - Nettoyer un effet avec une fonction de retour.
-

1. Qu'est-ce qu'un effet secondaire ?

Un **effet secondaire** est une opération qui interagit avec l'extérieur ou qui sort du cycle normal de rendu :

- Appels réseau (fetch, axios...)
- Accès au `localStorage`
- Manipulation du `document`
- Gestion de timers ou d'événements globaux

Ces opérations ne doivent **pas être déclenchées directement dans le rendu**, mais dans un hook `useEffect`.

2. Utiliser `useEffect` au montage

Le hook `useEffect` permet d'exécuter une fonction **une fois après le premier rendu**.

- C'est l'équivalent de `componentDidMount` dans les composants de classe.
- On utilise un tableau de dépendances vide (`[]`).

Exemple : affichage d'un message au montage

```
import {useEffect} from "react";

function Welcome() {
  useEffect(() => {
    console.log("Composant monté !");
  }, []);

  return <p>Bienvenue !</p>;
}
```

3. Réagir à un changement de state

On peut aussi utiliser `useEffect` pour déclencher une action à chaque fois qu'une **valeur change**.

- On passe cette valeur dans le tableau de dépendances.

Exemple : déclencher une action après une mise à jour

```
import {useState, useEffect} from "react";

function ClickTracker() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log(`Le bouton a été cliqué ${count} fois`);
  }, [count]);

  return <button onClick={() => setCount(count + 1)}>Cliquez-moi</button>;
}
```

4. Nettoyer un effet

Quand on utilise un `setInterval`, un écouteur d'événement global, ou un effet persistant, on peut renvoyer une fonction de nettoyage dans le `useEffect`.

- Cette fonction sera appelée **avant le démontage du composant**, ou **avant la relance de l'effet**.

Exemple : minuterie avec nettoyage

```
import {useState, useEffect} from "react";

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);

    return () => {
      clearInterval(id);
    };
  }, []);

  return <p>Temps écoulé : {seconds}s</p>;
}
```

5. Exercice de mise en pratique

Consignes

1. Créez un composant `Clock` qui :
 - Affiche l'heure actuelle (ex : 13:42:15).
 - Met à jour l'heure toutes les secondes avec `setInterval`.
 - Utilise `useEffect` pour lancer l'intervalle au montage.
 - Nettoie proprement l'intervalle au démontage.
2. Intégrez `Clock` dans un composant `App`.

Organisation attendue

```
/exercices
├── 12-react-clock-useeffect
│   ├── App.jsx
│   ├── Clock.jsx
│   └── main.jsx
```